Implementación del proyecto aplicado “SeparaBien”: solución digital para la gestión integral de residuos sólidos en la Ciudadela San Eduardo, Popayán

Fase 3 – Implementación del proyecto

Daniliz Fernando Gómez Pinto

Curso 202047922A_2204 – Trabajo de Grado

Tutora: María Consuelo Rodríguez

Universidad Nacional Abierta y a Distancia – UNAD

Escuela de Ciencias Básicas, Tecnología e Ingeniería – ECBTI

Tecnología en Desarrollo de Software

Popayán, octubre de 2026

Tabla de contenido

Planteamiento del problema....3
Línea de investigación....3
Justificación....4
Objetivos....4
Marco conceptual y teórico....5
Metodología....8
Cronograma....9
Recursos necesarios....10
Resultados esperados....10
Análisis y diseño....11
Implementación (codificación o prototipos)....20
Pruebas y validación....25
Referencias bibliográficas....30
Bibliografía....31

Planteamiento del problema

La Ciudadela San Eduardo, Etapa II, en Popayán, Cauca, presenta una deficiente separación de residuos sólidos en la fuente. A pesar de la existencia del Plan de Gestión Integral de Residuos Sólidos (PGIRS) de Popayán 2016-2027 (Alcaldía de Popayán, 2016), se evidencian fallas en la práctica cotidiana de la comunidad. Siguiendo el esquema de Narváez (2016), se identifica lo siguiente:

Síntomas. Acumulación de basura en horarios no establecidos, mezcla de residuos aprovechables con no aprovechables y presencia de malos olores en las zonas de acopio.

Causas. Desconocimiento de los horarios de la ruta de Urbaser Popayán, falta de educación ambiental práctica y ausencia de herramientas digitales locales que guíen el proceso de separación.

Pronóstico. De no intervenir, se agravará la contaminación, aumentarán las quejas vecinales y se dificultará el trabajo de los recicladores de oficio, perpetuando el bajo índice de aprovechamiento nacional (17 %) [citar la fuente de este dato].

Control al pronóstico (propuesta). Desarrollar “SeparaBien”, una solución digital (aplicación web progresiva) que eduque, notifique y gamifique el proceso de separación, empoderando a la comunidad con información clara y accesible.

Pregunta orientadora. ¿Cómo puede una solución digital ligera y centrada en el usuario mejorar la separación de residuos en la fuente entre los habitantes de la Ciudadela San Eduardo?

Línea de investigación

Ingeniería de software y tecnologías de la información, con énfasis en el desarrollo de soluciones digitales centradas en el usuario y la sostenibilidad comunitaria. Esta línea se alinea con los Objetivos de Desarrollo Sostenible (ODS) 9 (Industria, innovación e infraestructura) y 11 (Ciudades y comunidades sostenibles) de la Agenda 2030 (Naciones Unidas, 2015), aplicando el ciclo de vida del desarrollo de software para resolver problemáticas reales del entorno, como la gestión integral de residuos sólidos en la Ciudadela San Eduardo, Popayán.

Justificación

Este proyecto responde a una necesidad real de la comunidad de Popayán y se alinea con los ODS 9 y 11 de la Agenda 2030 (Naciones Unidas, 2015). Tomando como referente a León et al. (2017), quienes destacan el valor de la información científica y tecnológica para la investigación y la innovación, la propuesta no busca reinventar la normativa, sino crear un puente de innovación práctica que traduzca el PGIRS a un lenguaje ciudadano, simple y digital.

Al complementar la gestión actual con una herramienta centrada en el usuario, se mejora la salud pública del barrio, se facilita el trabajo de los recicladores y se aplica de manera integral el conocimiento adquirido en el programa de Tecnología en Desarrollo de Software, desde el análisis de requisitos hasta el diseño, la implementación y la prueba de una solución real.

Objetivos

Objetivo general

Desarrollar un producto mínimo viable (MVP) de la solución digital “SeparaBien”, que optimice la gestión y separación de residuos sólidos en la Ciudadela San Eduardo mediante herramientas interactivas, notificaciones y gamificación.

Objetivos específicos

- Diagnosticar las necesidades y los hábitos de separación de residuos de los habitantes mediante técnicas de recolección de datos validadas (observación, encuestas y entrevistas).
- Diseñar un prototipo funcional (MVP) de la aplicación aplicando principios de Lean Startup, para validar su utilidad con la Junta de Acción Comunal (JAC) y los vecinos.
- Integrar mecanismos de gamificación (como un quiz educativo interactivo e insignias) para incentivar el aprendizaje y la participación continua de los usuarios.
- Evaluar la usabilidad del MVP con vecinos voluntarios, para identificar mejoras y documentar los resultados.

Marco conceptual y teórico

A continuación se presentan los conceptos, teorías y referentes que fundamentan el desarrollo del proyecto “SeparaBien”, articulando la problemática ambiental comunitaria con las metodologías de desarrollo de software aplicadas.

Gestión integral de residuos sólidos (GIRS)

La GIRS es el conjunto de operaciones y disposiciones encaminadas a dar a los residuos generados el destino más adecuado desde el punto de vista ambiental, según su naturaleza, características, efectos y destino final (Ministerio de Ambiente y Desarrollo Sostenible, 2016). En Colombia, esta gestión se articula a través de los PGIRS, instrumentos adoptados por los municipios para planificar la prestación del servicio público de aseo. Para facilitar la separación en la fuente se adoptó un código de colores: caneca blanca para residuos aprovechables, negra para no aprovechables y verde para orgánicos aprovechables (Ministerio de Ambiente y Desarrollo Sostenible, 2019).

Para Popayán se toma como referente el PGIRS 2016-2027 (Alcaldía de Popayán, 2016), que establece lineamientos para la separación en la fuente, la recolección diferenciada y el aprovechamiento de residuos, y delega la operación en el concesionario Urbaser Popayán S.A. E.S.P. Este documento es la base sobre la cual se construye el contenido educativo de la aplicación.

Objetivos de Desarrollo Sostenible (ODS)

El proyecto se alinea con la Agenda 2030 de las Naciones Unidas (2015), específicamente con dos objetivos. El ODS 9 (Industria, innovación e infraestructura) se materializa en el uso de tecnología accesible (PWA) para educar y conectar a la comunidad, mediante una solución ligera, escalable y de bajo costo. El ODS 11 (Ciudades y comunidades sostenibles) se refleja en la mejora de la gestión ambiental del barrio, la reducción de residuos enviados a rellenos sanitarios y el fortalecimiento del tejido social.

Design Thinking

El Design Thinking se entiende no solo como un proceso empresarial, sino como una mentalidad centrada en el ser humano para resolver problemas complejos o «retorcidos» (Clarke, 2020, pp. 15-18). Desde esta mirada, la empatía y la correcta definición del problema son los cimientos que deben establecerse antes de intentar idear cualquier solución técnica. El proceso suele organizarse en cinco fases: empatizar, definir, idear, prototipar y testear.

En “SeparaBien” se aplica para comprender las barreras reales de los habitantes al separar sus residuos, mediante observación directa de las zonas de acopio, entrevistas semiestructuradas con líderes de la JAC y encuestas digitales a los vecinos para validar hábitos de reciclaje.

Lean Startup

Llamas Fernández (2018) presenta el Lean Startup como un modelo que prioriza el aprendizaje validado sobre la planificación exhaustiva y la intuición. Su ciclo crear-medir-aprender se apoya en el producto mínimo viable (MVP) para probar hipótesis con el menor costo posible, y exige decidir cuándo pivotar (cambiar de estrategia) o perseverar, con el fin de reducir la incertidumbre y el desperdicio de recursos. En “SeparaBien” se aplica así:

- **Construir:** se toma como producto mínimo viable la cartilla digital interactiva ya desarrollada, que contiene la guía de clasificación, el calendario de recolección y consejos prácticos.
- **Medir:** se valida el MVP con 5 a 10 vecinos voluntarios, midiendo comprensión de la información, facilidad de uso y frecuencia de consulta.
- **Aprender:** con la retroalimentación se itera hacia una PWA que incorpore notificaciones, quiz educativo y, en una etapa posterior, un mapa de puntos de acopio.
- **Pivotar o perseverar:** con los resultados de la prueba de usabilidad se decide si se mantiene la estrategia actual o se cambia (por ejemplo, ajustando contenidos o prioridades).

Gamificación

Villegas et al. (2018) analizan la relación entre la experiencia de usuario (UX) y el diseño de juegos, y proponen la metodología I'm In, en la que la gamificación es una herramienta estratégica que va más allá del entretenimiento: busca modificar el comportamiento del usuario, aumentar su motivación intrínseca y mejorar su retención. [resumir en una frase las fases de la metodología I'm In que usted leyó]. En "SeparaBien" se implementa mediante:

- **Quiz educativo interactivo:** pequeñas rondas donde el usuario identifica en qué caneca va cada residuo y gana puntos por respuestas correctas.
- **Insignias de logro:** reconocimientos simbólicos por completar metas (por ejemplo, "Vecino Reciclador" o "Experto en Orgánicos").
- **Rachas de uso:** incentivo por jugar de forma continua, día tras día.

Metodologías ágiles

Las metodologías ágiles se basan en el Manifiesto por el Desarrollo Ágil de Software (Beck et al., 2001), que valora a las personas y su interacción, el software funcionando, la colaboración con el cliente y la respuesta al cambio. En este proyecto se adopta un enfoque ágil ligero, inspirado en Scrum y Kanban: ciclos de trabajo (sprints) semanales con objetivos claros y un tablero de tareas (por hacer, en curso, hecho) para el seguimiento del avance.

Aplicaciones web progresivas (PWA) y usabilidad

Una PWA es una aplicación web que, mediante tecnologías estándar como el manifiesto web y los *service workers*, ofrece una experiencia similar a la de una aplicación móvil: puede instalarse, funcionar parcialmente sin conexión y enviar notificaciones (Mozilla, s. f.). Esto reduce costos frente a una aplicación nativa y facilita su adopción en la comunidad.

La usabilidad, según Nielsen (1993), se evalúa en términos de facilidad de aprendizaje, eficiencia, facilidad para recordar, tasa de errores y satisfacción del usuario. Estos criterios orientan las pruebas con los vecinos.

Requisitos, modelado y pruebas de software

Sommerville (2011) distingue los requerimientos funcionales, que describen lo que el sistema debe hacer, de los no funcionales, que establecen cualidades y restricciones como la usabilidad, la disponibilidad o el rendimiento. Para representar el diseño se utiliza el Lenguaje Unificado de Modelado (UML) (Booch et al., 2006), mediante diagramas de casos de uso, de clases y de flujo. En cuanto a las pruebas, las de unidad verifican cada componente por separado, las de integración comprueban que los módulos funcionen juntos y las de validación confirman que el sistema cumple los requisitos del usuario (Sommerville, 2011).

Metodología

El proyecto integra Design Thinking, Lean Startup y gamificación dentro de un enfoque ágil, organizado en cinco etapas:

- **Etapa 1 – Empatizar y definir (Design Thinking):** observación de campo en las zonas de acopio, entrevistas semiestructuradas a líderes de la JAC y una encuesta digital a 20-30 vecinos. Como indica Asti Vera (2015), esta etapa busca comprender las necesidades reales y validar los instrumentos de recolección de datos; y desde el Design Thinking, la empatía y la definición correcta del problema van antes de proponer cualquier solución (Clarke, 2020, pp. 15-18).
- **Etapa 2 – Idear y prototipar:** a partir de las necesidades encontradas, se levantan los requerimientos, se modela el diseño y se elaboran los wireframes, con interfaz clara, alto contraste y lenguaje sencillo, siguiendo principios de usabilidad.
- **Etapa 3 – Construir el MVP (Lean Startup y gamificación):** se toma como base la cartilla digital interactiva y se evoluciona hacia una PWA con guía de clasificación, calendario de recolección y quiz educativo con insignias. Según Llamas Fernández (2018), el MVP permite probar hipótesis con el menor costo posible. Los puntos, las insignias y las rachas se incorporan para motivar y retener al usuario (Villegas et al., 2018).

- **Etapa 4 – Medir y aprender:** pruebas de unidad, integración y validación, y pruebas de usabilidad con 5 a 10 vecinos voluntarios, recolectando su opinión mediante un cuestionario digital.
- **Etapa 5 – Iterar y documentar:** con base en la retroalimentación se decide si perseverar o pivotar (Llamas Fernández, 2018), se ajusta el MVP y se elabora la documentación final, procesando la información como sugiere Lerma González (2009).

Tecnologías a utilizar

- **Frontend:** HTML5, CSS3 y JavaScript, para garantizar ligereza y compatibilidad.
- **Diseño UI/UX:** Figma, para el prototipado.
- **Despliegue:** GitHub Pages, para un alojamiento gratuito y accesible.
- **Seguimiento ágil:** tablero Kanban (por ejemplo, GitHub Projects).
- **Base de datos (fase futura):** Firebase o Supabase, para gestionar usuarios y puntuaciones del quiz.

Cronograma

El cronograma se ajusta a la duración de la Fase 3: del 4 de octubre al 5 de noviembre de 2026, con 45 horas de trabajo independiente. Se organiza en cinco semanas, una por cada sprint, y cubre las cinco etapas de la metodología.

Tabla 1

Cronograma de actividades de la Fase 3

Semana	Etapa	Actividades principales	Resultados esperados
1 (4-10 oct)	Etapa 1	Ajuste de la propuesta con la retroalimentación del tutor y levantamiento de requerimientos funcionales y no funcionales.	Requerimientos definidos y priorizados.
2 (11-17 oct)	Etapa 2	Modelado: casos de uso, clases, flujos, arquitectura y modelo de datos; wireframes de las pantallas.	Diagramas de análisis y diseño.

Semana	Etapa	Actividades principales	Resultados esperados
3 (18-24 oct)	Etapa 3	Codificación del MVP como PWA: buscador, calendario, quiz, logros y modo sin conexión (Lean Startup y gamificación).	MVP funcional.
4 (25-31 oct)	Etapa 4	Pruebas de unidad, integración y validación; pruebas de usabilidad con 5 a 10 vecinos.	Registro de pruebas, fallos y correcciones.
5 (1-5 nov)	Etapa 5	Correcciones finales, documentación, publicación en el foro de la Fase 3 y entrega del PDF.	Documento final entregado.

Nota. Elaboración propia, con las fechas de la guía de la Fase 3.

Recursos necesarios

- **Técnicos:** computador personal, conexión a internet, VS Code, Figma, Node.js (para ejecutar las pruebas automáticas), un navegador moderno y cuenta de GitHub.
- **Humanos:** estudiante desarrolladora, líderes de la JAC (como validadores), vecinos voluntarios y tutora.
- **De información:** PGIRS de Popayán 2016-2027 y datos de las rutas de Urbaser Popayán.

Resultados esperados

- Un documento de diagnóstico comunitario con datos confiables sobre los hábitos de reciclaje en la ciudadela.
- Un prototipo funcional (MVP) de “SeparaBien” desplegado en la web, con guía de clasificación, calendario y quiz educativo.
- Evidencia de validación del MVP mediante pruebas de usabilidad con la comunidad.
- Documentación completa del proceso bajo normas APA.

Análisis y diseño

En esta sección se describen los requerimientos del sistema y se modela su diseño con diagramas UML (Booch et al., 2006). El modelo corresponde a la versión actual de "SeparaBien", un prototipo funcional que se ejecuta completamente en el dispositivo del usuario.

Requerimientos funcionales

Tabla 2

Requerimientos funcionales

ID	Descripción	Prioridad
RF-01	El sistema permite buscar un residuo por nombre o palabra clave (sin importar tildes ni mayúsculas) y muestra la caneca donde va y cómo prepararlo.	Alta
RF-02	El vecino indica qué tipo de residuo se recoge cada día de la semana y el sistema muestra la próxima recolección.	Alta
RF-03	El sistema arma rondas de 5 preguntas distintas sobre dónde va cada residuo.	Alta
RF-04	Al responder, el sistema da retroalimentación inmediata, explica dónde va el residuo y suma 10 puntos por cada acierto.	Alta
RF-05	El sistema lleva una racha de días consecutivos en los que el vecino juega.	Media
RF-06	El sistema otorga insignias (Primer acierto, Vecino Reciclador, Experto en Orgánicos y Racha de 3 días) y muestra los logros del vecino.	Media
RF-07	Si el vecino lo autoriza, el sistema envía un recordatorio la víspera de la recolección.	Media
RF-08	El sistema guarda en el dispositivo el calendario, los puntos y las insignias, y los recupera al volver a abrir la app.	Alta

Nota. Elaboración propia.

Requerimientos no funcionales

Tabla 3

Requerimientos no funcionales (características de calidad)

ID	Categoría	Descripción
RNF-01	Usabilidad	Lenguaje sencillo, una tarea principal por pantalla y botones de al menos 44 px de alto.
RNF-02	Disponibilidad	La app funciona sin conexión a internet después de la primera visita.

ID	Categoría	Descripción
RNF-03	Ligereza	Sin librerías ni recursos externos; el peso total de los archivos es menor a 100 KB (actualmente 40,4 KB).
RNF-04	Compatibilidad	Funciona en navegadores modernos y en pantallas desde 320 px de ancho, sin desplazamiento horizontal.
RNF-05	Privacidad	No pide registro ni datos personales; la información queda solo en el dispositivo.
RNF-06	Accesibilidad	Foco visible con teclado, campos con etiqueta, contraste alto y respeto de la preferencia de movimiento reducido.
RNF-07	Mantenibilidad	La lógica está separada de la interfaz y cubierta por pruebas automáticas.

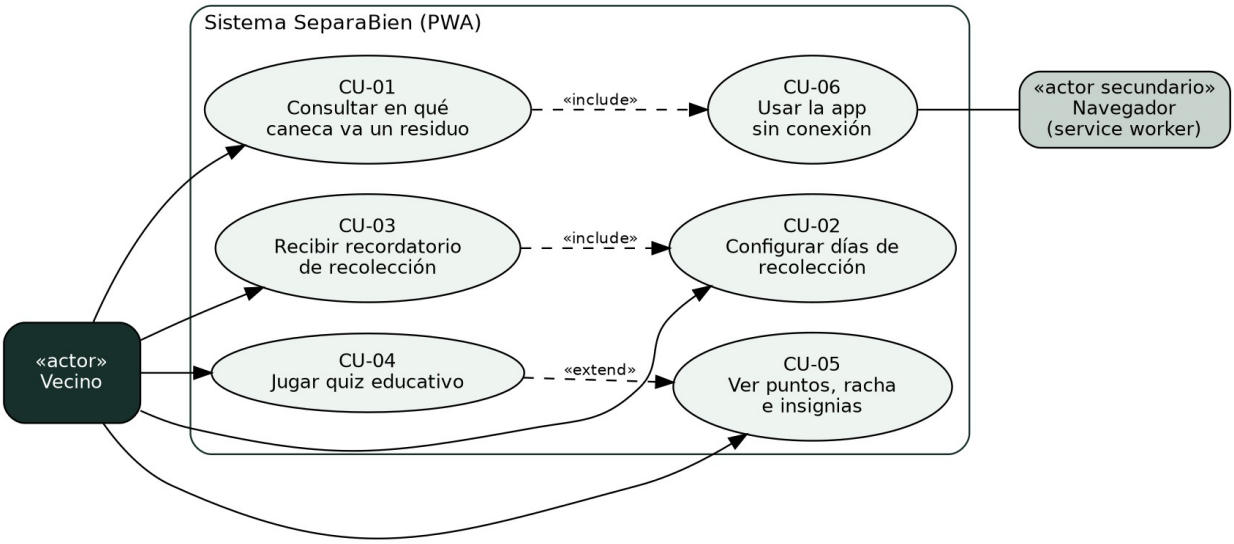
Nota. Elaboración propia.

Diagrama de casos de uso

El único actor humano es el vecino. El navegador participa como actor secundario porque el service worker permite el uso sin conexión.

Figura 1

Diagrama de casos de uso de SeparaBien



Nota. Elaboración propia.

Tabla 4

Descripción de los casos de uso

Caso	Actor	Flujo principal	Flujo alterno
CU-01	Vecino	Escribe un residuo; el sistema muestra la caneca y la nota de preparación.	Si no hay coincidencias, muestra un mensaje con ejemplos.
CU-02	Vecino	Abre Calendario y elige el tipo de recolección de cada día; el sistema guarda y actualiza el aviso de Inicio.	Si no configura nada, Inicio lo invita a hacerlo.
CU-03	Vecino	Pulsa «Activar recordatorio» y acepta el permiso; al abrir la app la víspera recibe la notificación.	Si rechaza el permiso o el navegador no lo soporta, se informa y la app sigue funcionando.
CU-04	Vecino	Pulsa «Empezar», responde 5 preguntas y ve su resultado.	Si responde mal, el sistema explica cuál era la caneca correcta.
CU-05	Vecino	Abre Logros y ve puntos, aciertos, racha e insignias.	Las insignias no ganadas aparecen bloqueadas.
CU-06	Vecino y navegador	La app se guarda en el dispositivo en la primera visita y abre sin conexión.	Si el navegador no admite service workers, la app funciona solo con conexión.

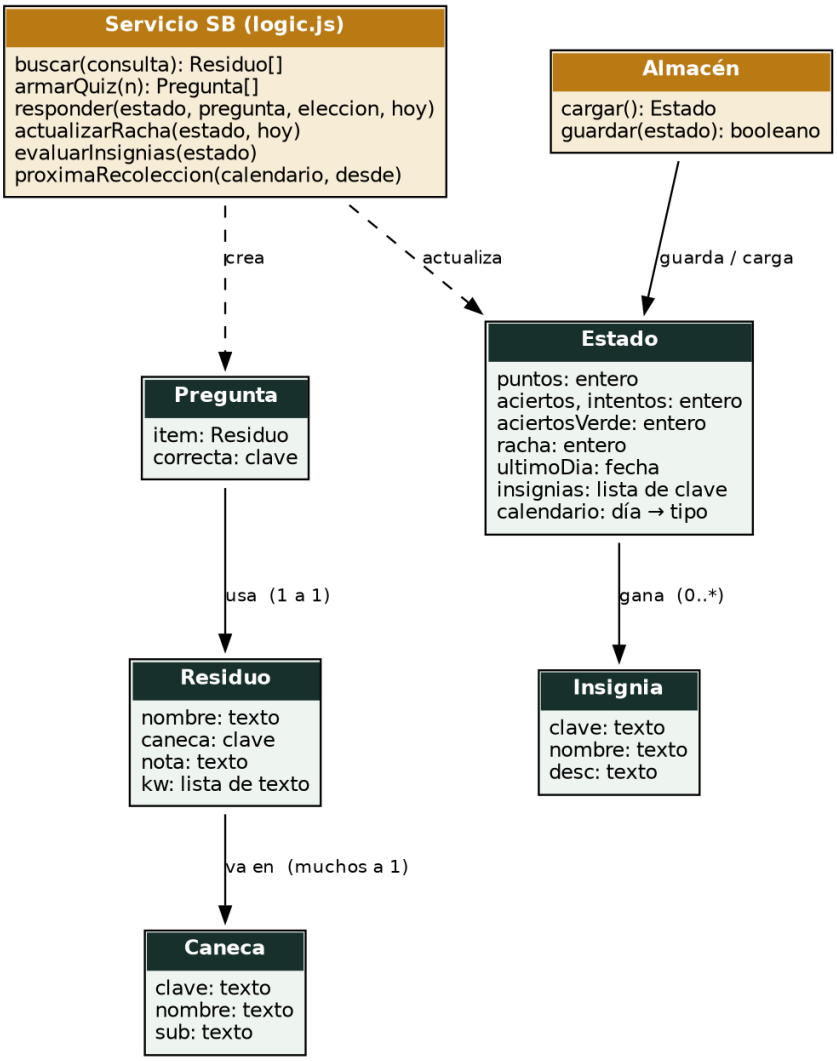
Nota. Elaboración propia.

Diagrama de clases

El diagrama muestra las entidades del sistema y los servicios que las manipulan. Las clases Caneca, Residuo, Pregunta, Insignia y Estado representan los datos; el Servicio SB (archivo logic.js) concentra las reglas de negocio y el Almacén guarda y recupera el Estado en el dispositivo.

Figura 2

Diagrama de clases de SeparaBien



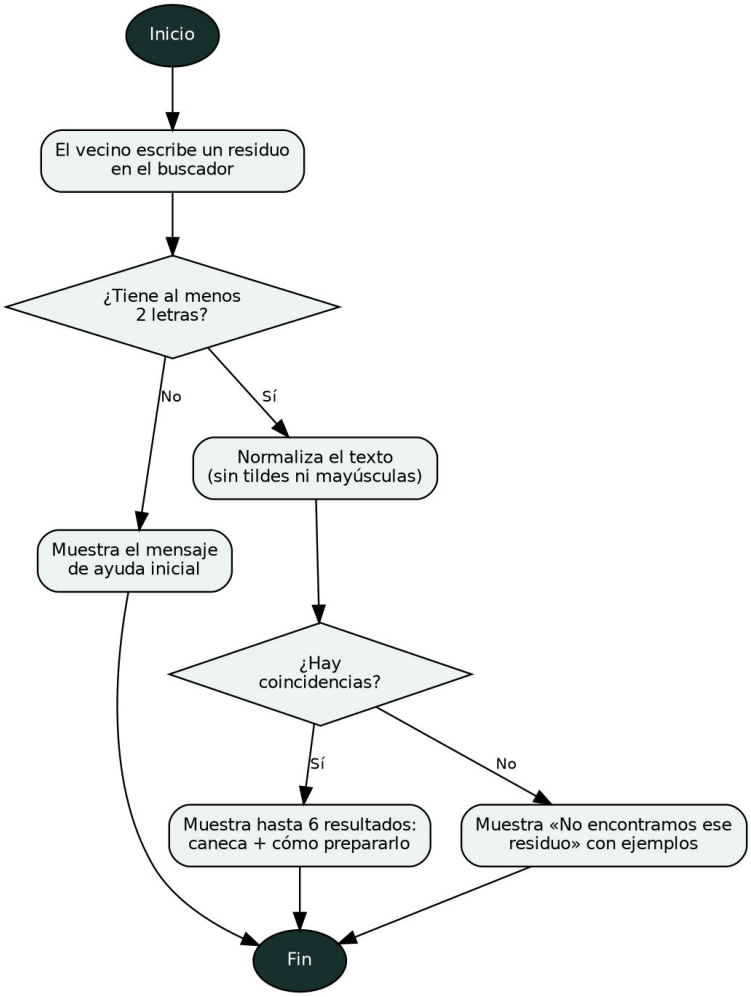
Nota. Elaboración propia.

Diagramas de flujo

Se modelaron los dos procesos principales: la consulta de un residuo y la ronda del quiz.

Figura 3

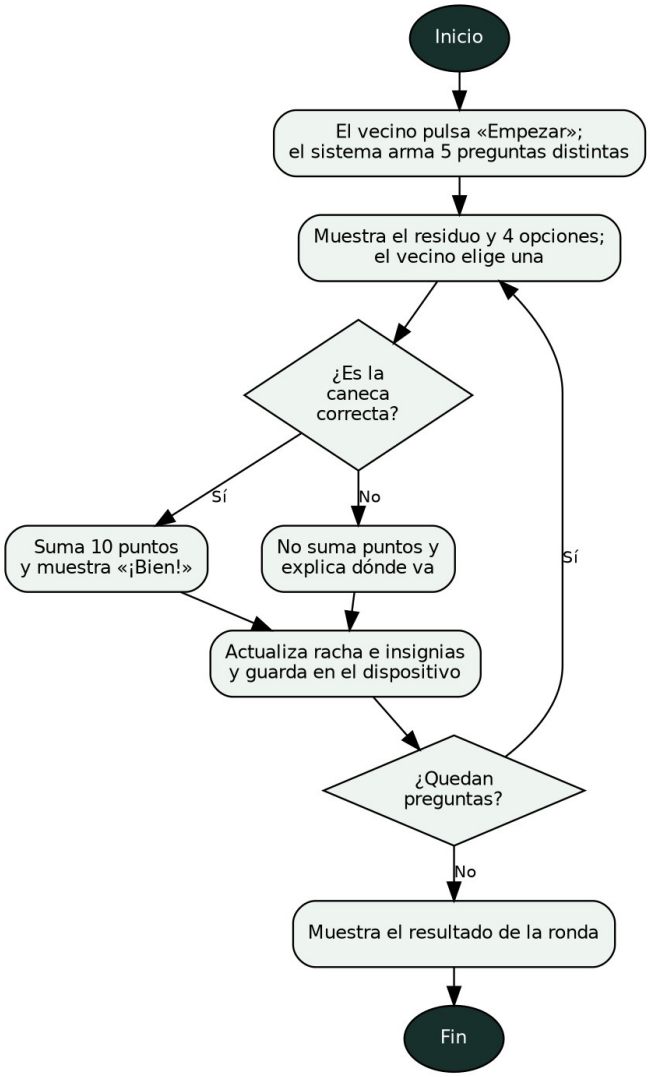
Diagrama de flujo de la búsqueda de un residuo



Nota. Elaboración propia.

Figura 4

Diagrama de flujo de una ronda del quiz



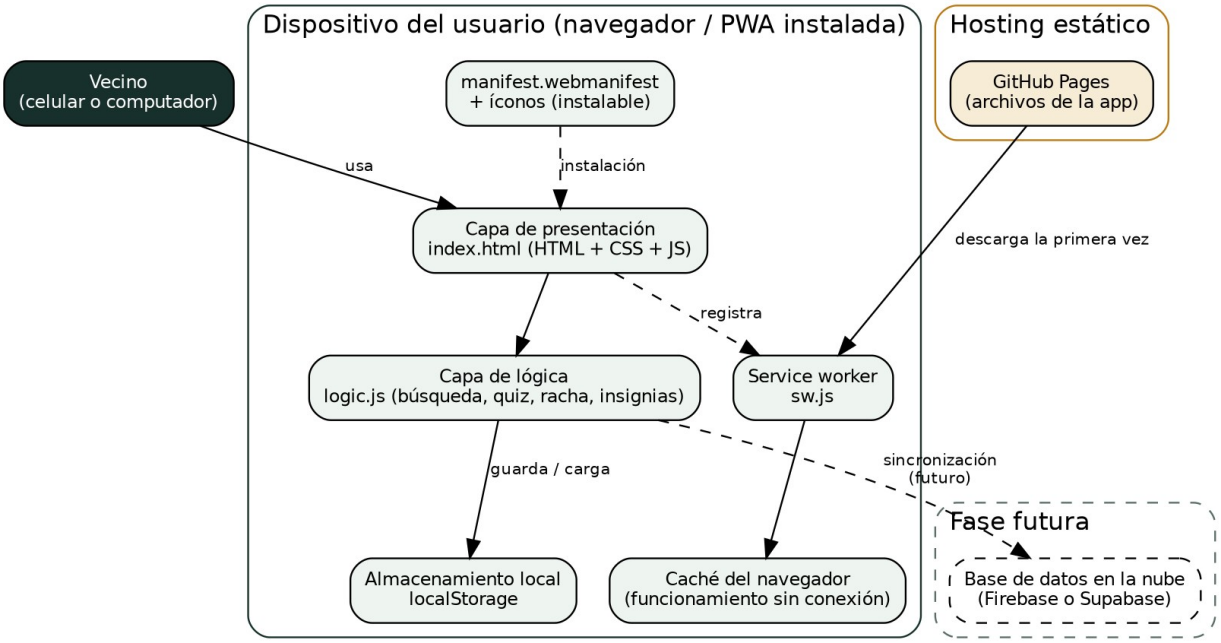
Nota. Elaboración propia.

Arquitectura

"SeparaBien" es una aplicación web progresiva (PWA) de tres capas que se ejecutan en el dispositivo: presentación (index.html), lógica (logic.js) y almacenamiento local (localStorage). El service worker guarda los archivos en la caché del navegador para el uso sin conexión, y el manifiesto permite instalar la app en la pantalla de inicio. Los archivos se publican como sitio estático en GitHub Pages, por lo que no se requiere servidor propio. La base de datos en la nube (Firebase o Supabase) queda como fase futura.

Figura 5

Arquitectura de SeparaBien



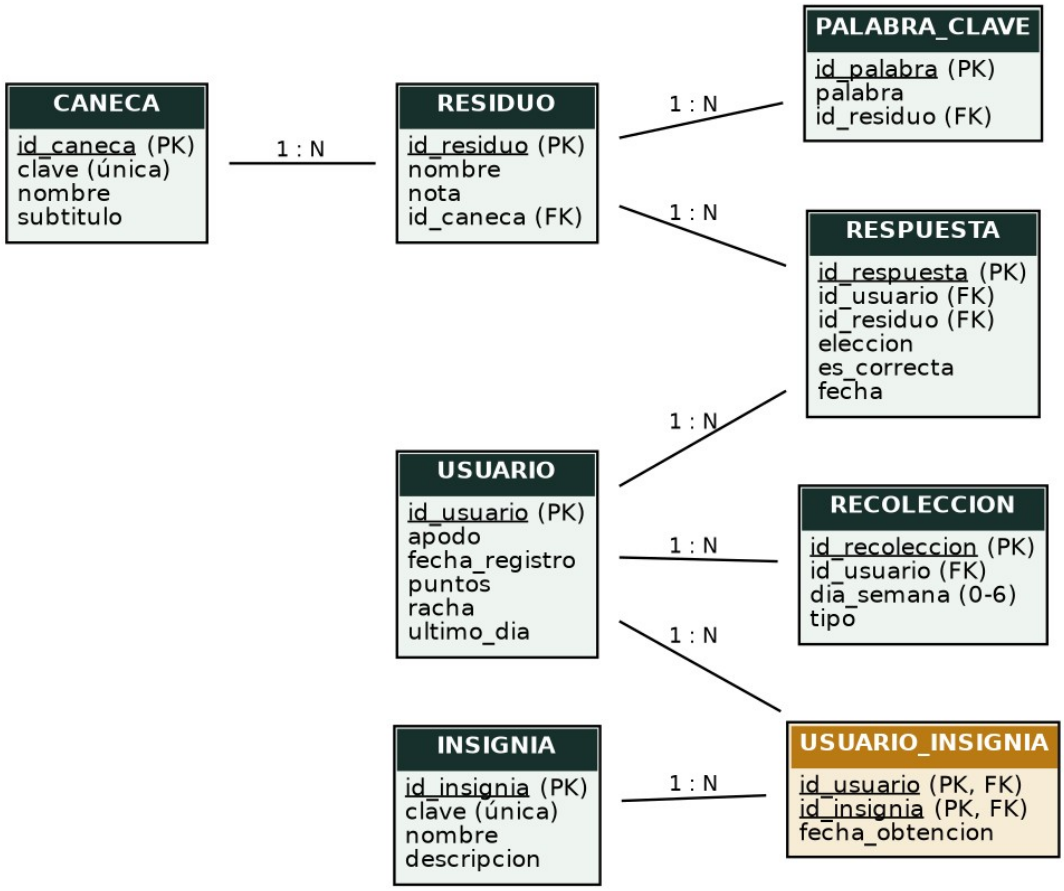
Nota. Elaboración propia.

Base de datos

En el prototipo actual, los datos del vecino se guardan como un objeto JSON en localStorage (clave separabien:estado), con los campos puntos, aciertos, intentos, aciertosVerde, racha, ultimoDia, insignias y calendario. El catálogo de residuos, canecas e insignias va incluido en el código. Para la fase futura, con sincronización en la nube, se diseñó el modelo relacional de la Figura 6, que incorpora tablas, relaciones y claves.

Figura 6

Modelo entidad-relación para la fase futura



Nota. Elaboración propia.

Tabla 5

Tablas, claves primarias y claves foráneas del modelo relacional

Tabla	Clave primaria	Claves foráneas	Descripción
USUARIO	id_usuario	—	Vecino que usa la app y su progreso general.
CANECA	id_caneca	—	Canecas (blanca, negra, verde) y punto especial.
RESIDUO	id_residuo	id_caneca	Catálogo de residuos y su nota de preparación.
PALABRA_CLAVE	id_palabra	id_residuo	Palabras para encontrar un residuo.

Tabla	Clave primaria	Claves foráneas	Descripción
RESPUESTA	id_respuesta	id_usuario, id_residuo	Cada respuesta dada en el quiz.
RECOLECCION	id_recoleccion	id_usuario	Tipo de recolección de cada día de la semana.
INSIGNIA	id_insignia	—	Catálogo de insignias.
USUARIO_INSIGNIA	id_usuario + id_insignia	id_usuario, id_insignia	Insignias ganadas por cada vecino.

Nota. Elaboración propia.

Implementación (codificación o prototipos)

El producto desarrollado es un **prototipo funcional** (MVP): las cuatro secciones de la app (Inicio, Calendario, Quiz y Logros) funcionan, guardan datos y abren sin conexión. Se programó con HTML, CSS y JavaScript sin librerías externas, siguiendo principios de desarrollo ágil: el trabajo se hizo en ciclos cortos, probando cada función antes de pasar a la siguiente.

Estructura del proyecto

Tabla 6

Archivos del proyecto SeparaBien

Archivo	Función	Tamaño
index.html	Interfaz: las cuatro secciones, los estilos y la conexión con la lógica.	14,6 KB
logic.js	Lógica: catálogo, búsqueda, quiz, puntos, racha, insignias, calendario y almacenamiento.	9,8 KB
sw.js	Service worker: guarda la app en el dispositivo para usarla sin conexión.	0,7 KB
manifest.webmanifest	Datos para instalar la app en la pantalla de inicio.	0,5 KB
icon-192.png, icon-512.png	Íconos de la aplicación.	6,5 KB y 8,3 KB
tests/test_logic.js	Pruebas automáticas de unidad e integración.	7,2 KB

Nota. Elaboración propia. Peso total de la app: 40,4 KB.

Código fuente comentado

Se presentan los fragmentos más importantes del código, resumidos. El código completo está en los archivos del proyecto.

Fragmento 1. Búsqueda de residuos (RF-01), en logic.js:

```
function buscar(consulta, items) {
  items = items || ITEMS;
  const q = normalizar(consulta);     // quita tildes y mayúsculas
  if (q.length < 2) return [];        // evita resultados con una sola letra
  const res = items.filter(it =>
    normalizar(it.nombre).includes(q) ||
    it.kw.some(k => normalizar(k).includes(q)));
  // los que empiezan con lo escrito aparecen primero
  return res.sort((a, b) =>
    Number(normalizar(b.nombre).startsWith(q)) -
    Number(normalizar(a.nombre).startsWith(q)));
```

```
}
```

Fragmento 2. Racha de días y registro de respuestas (RF-04 y RF-05), en logic.js:

```
function actualizarRacha(estado, hoy) {
  if (estado.ultimoDia === hoy) return estado;   // mismo día: no cambia
  const ayer = estado.ultimoDia &&
    (aFecha(hoy) - aFecha(estado.ultimoDia)) === 86400000;
  // sube si jugó ayer; si se saltó un día, vuelve a 1
  return Object.assign({}, estado,
    { racha: ayer ? estado.racha + 1 : 1, ultimoDia: hoy });
}

function responder(estado, pregunta, eleccion, hoy) {
  let e = actualizarRacha(estado, hoy);
  const acierto = eleccion === pregunta.correcta;
  e = Object.assign({}, e, {
    intentos: e.intentos + 1,
    aciertos: e.aciertos + (acierto ? 1 : 0),
    puntos: e.puntos + (acierto ? PUNTOS_ACIERTO : 0),   // +10 por acierto
    aciertosVerde: e.aciertosVerde +
      (acierto && pregunta.correcta === 'verde' ? 1 : 0)
  });
  // entrega solo las insignias que todavía no tenía
  const nuevas = evaluarInsignias(e).filter(k => !e.insignias.includes(k));
  e.insignias = e.insignias.concat(nuevas);
  return { estado: e, acierto, nuevas };
}
```

Fragmento 3. Uso sin conexión (RNF-02), en sw.js:

```
const VERSION = 'separabien-v1';
const ARCHIVOS = ['./', './index.html', './logic.js', /* ... */];

// al instalar, guarda los archivos en la caché del navegador
self.addEventListener('install', e => {
  e.waitUntil(caches.open(VERSION).then(c => c.addAll(ARCHIVOS)));
});

// al pedir un archivo, busca primero en la caché y si no está, va a la red
self.addEventListener('fetch', e => {
  e.respondWith(caches.match(e.request).then(r => r || fetch(e.request)));
});
```

Aplicación de las metodologías en el prototipo

- **Design Thinking:** el lenguaje de las pantallas es sencillo y cada residuo se explica con una nota práctica de cómo prepararlo, pensando en la persona que está en la cocina o en el punto de acopio (Clarke, 2020).

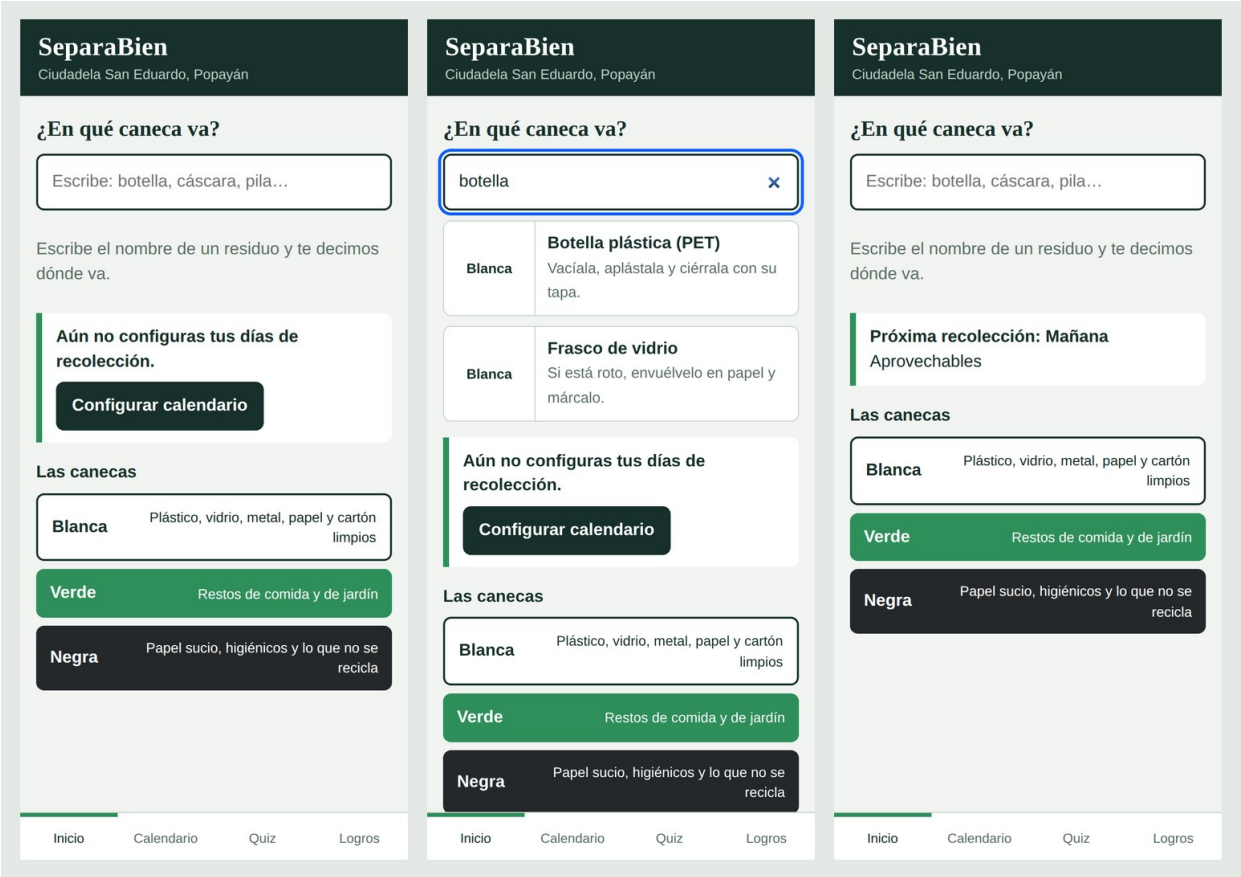
- **Lean Startup:** el MVP parte de la cartilla digital inicial y se amplía solo con lo necesario; la base de datos en la nube se aplaza hasta validar el uso real, y tras la prueba de usabilidad se decidirá si perseverar o pivotar (Llamas Fernández, 2018).
- **Gamificación:** el quiz entrega 10 puntos por acierto, la racha premia la constancia y las cuatro insignias reconocen los logros, con el fin de motivar y retener al usuario (Villegas et al., 2018).

Capturas de pantalla

Las siguientes capturas muestran las interfaces clave del prototipo funcional, tomadas en un navegador con tamaño de pantalla de celular (390 × 844 px).

Figura 7

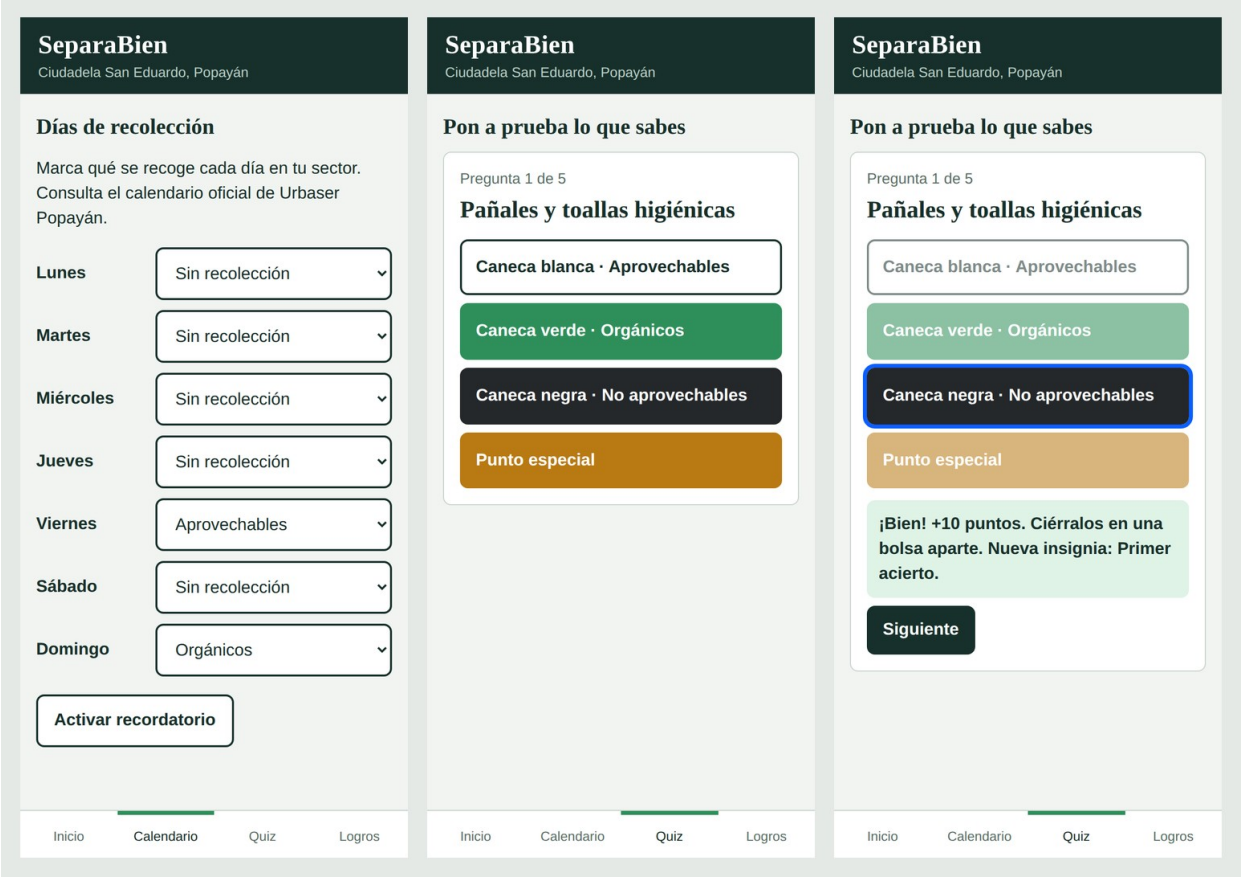
Pantalla de Inicio: buscador, aviso de recolección y canecas



Nota. De izquierda a derecha: Inicio sin búsqueda, resultados al buscar «botella» e Inicio con la próxima recolección configurada.

Figura 8

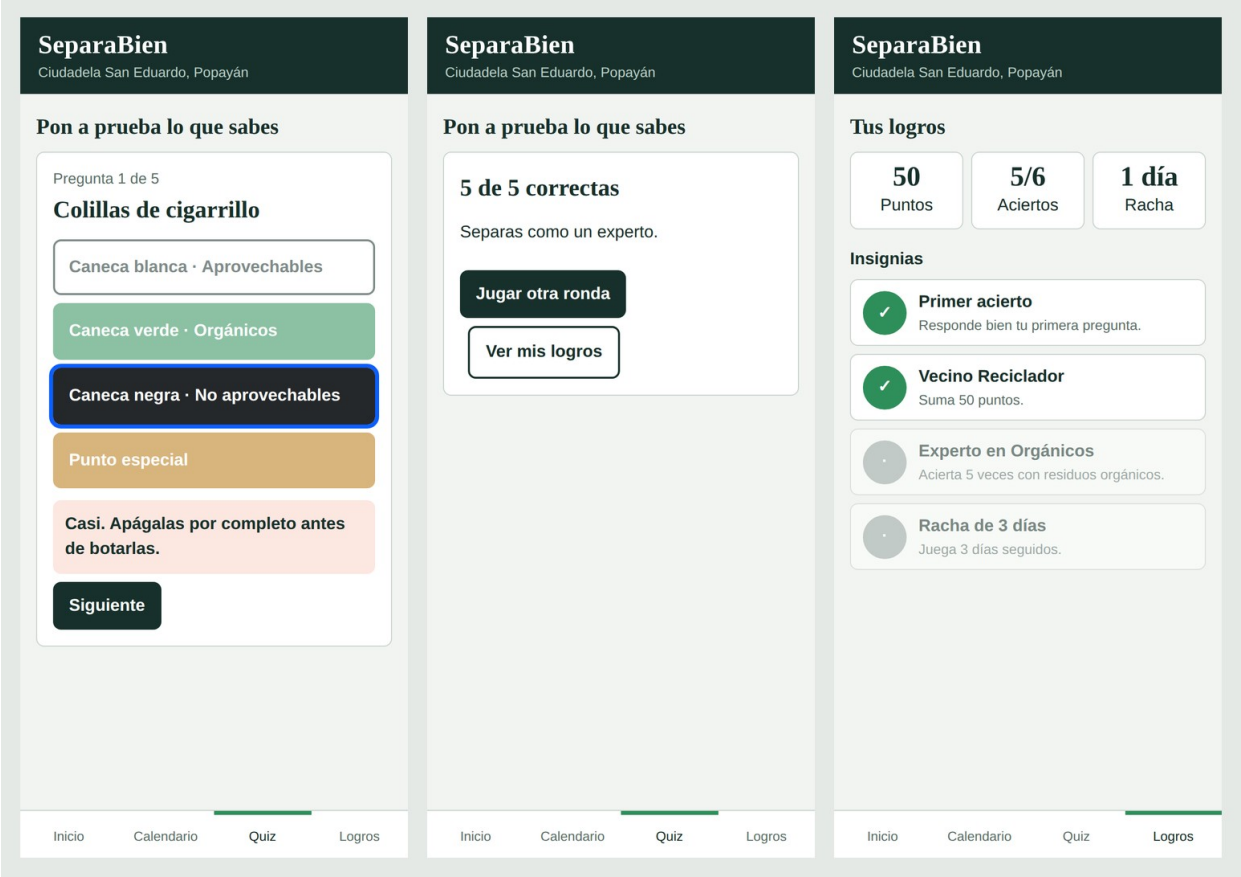
Calendario y quiz



Nota. De izquierda a derecha: configuración de días de recolección, pregunta del quiz y retroalimentación tras un acierto.

Figura 9

Error, resultado de la ronda y logros



Nota. De izquierda a derecha: retroalimentación tras una respuesta incorrecta, resultado de una ronda con 5 aciertos y pantalla de logros con insignias.

Publicación

La app se publica como sitio estático en GitHub Pages, copiando los archivos del proyecto al repositorio. Enlace de la aplicación publicada: [agregar el enlace de GitHub Pages cuando la publique].

Pruebas y validación

Siguiendo a Sommerville (2011), se aplicaron tres niveles de prueba: unidad, integración y validación. Las dos primeras se ejecutan con el archivo tests/test_logic.js usando Node.js; la validación se ejecutó en un navegador real (Chromium) simulando el uso de un vecino en un celular. Todos los resultados que se muestran provienen de ejecuciones reales de las pruebas.

Pruebas de unidad

Tabla 7

Resultados de las pruebas de unidad

ID	Prueba	Resultado
PU-01	buscar() encuentra «botella» y la manda a la caneca blanca	Aprobada
PU-02	buscar() ignora tildes y mayúsculas («CÁSCARA»)	Aprobada
PU-03	buscar() devuelve lista vacía con menos de 2 letras	Aprobada
PU-04	buscar() devuelve lista vacía si el residuo no existe	Aprobada
PU-05	buscar(«pila») clasifica como punto especial	Aprobada
PU-06	armarQuiz(5) devuelve 5 preguntas sin repetir	Aprobada
PU-07	armarQuiz con la misma semilla da el mismo orden (reproducible)	Aprobada
PU-08	responder() suma 10 puntos con respuesta correcta	Aprobada
PU-09	responder() no suma puntos con respuesta incorrecta	Aprobada
PU-10	La racha sube al jugar días consecutivos	Aprobada
PU-11	La racha se reinicia a 1 si se salta un día	Aprobada
PU-12	La racha no cambia si se juega dos veces el mismo día	Aprobada
PU-13	La racha funciona al cambiar de mes (31 oct → 1 nov)	Aprobada
PU-14	Se otorga «Vecino Reciclador» al llegar a 50 puntos	Aprobada
PU-15	Una insignia ya ganada no se entrega dos veces	Aprobada
PU-16	Se otorga «Experto en Orgánicos» tras 5 aciertos en verde	Aprobada
PU-17	proximaRecoleccion() encuentra el siguiente día configurado	Aprobada
PU-18	proximaRecoleccion() devuelve null si no hay días configurados	Aprobada
PU-19	Todos los residuos del catálogo tienen una caneca válida	Aprobada

Nota. Resultado: 19 de 19 pruebas aprobadas.

Pruebas de integración

Tabla 8

Resultados de las pruebas de integración

ID	Prueba	Resultado
PI-01	Jugar una ronda y guardar: al recargar se conservan puntos e insignias	Aprobada
PI-02	Calendario configurado se guarda y la próxima recolección se calcula tras recargar	Aprobada
PI-03	Búsqueda y quiz son coherentes: cada pregunta coincide con el resultado del buscador	Aprobada
PI-04	Si el almacenamiento falla o está dañado, la app arranca con estado inicial	Aprobada

Nota. Resultado: 4 de 4 pruebas aprobadas.

Pruebas de validación de requisitos

Estas pruebas recorren la app completa en un navegador (carga, búsqueda, calendario, quiz, logros, recarga, uso sin conexión y pantallas pequeñas) para confirmar que cumple los requerimientos.

Tabla 9

Resultados de las pruebas de validación (última ejecución)

ID	Prueba	Resultado
PV-01	La app carga y muestra el título «SeparaBien»	Aprobada
PV-02	Buscar «botella» muestra la caneca blanca	Aprobada
PV-03	Buscar «pila» muestra «Punto especial»	Aprobada
PV-04	Un residuo inexistente muestra un mensaje de ayuda	Aprobada
PV-05	Sin calendario, Inicio invita a configurarlo	Aprobada
PV-06	Con recolección mañana, Inicio muestra «Mañana» y el tipo	Aprobada
PV-07	Una respuesta correcta muestra «¡Bien! +10 puntos»	Aprobada
PV-08	Tras responder, los botones se bloquean	Aprobada
PV-09	Al terminar muestra «5 de 5 correctas»	Aprobada
PV-10	Una respuesta incorrecta muestra «Casi» y explica dónde va	Aprobada
PV-11	Logros muestra 50 puntos y racha de 1 día	Aprobada
PV-12	Se desbloquean las insignias «Primer acierto» y «Vecino Reciclador»	Aprobada

ID	Prueba	Resultado
PV-13	Al recargar la página se conservan puntos e insignias	Aprobada
PV-14	Al recargar se conserva el calendario configurado	Aprobada
PV-15	El service worker queda registrado	Aprobada
PV-16	Sin conexión a internet la app sigue abriendo	Aprobada
PV-17	Sin conexión el buscador sigue funcionando	Aprobada
PV-18	No hay desplazamiento horizontal en pantalla de 390 px	Aprobada
PV-19	Tampoco en pantalla pequeña de 320 px	Aprobada
PV-20	Sin errores en la consola del navegador	Aprobada

Nota. Resultado: 20 de 20 validaciones aprobadas.

Fallos encontrados y acciones correctivas

En la primera ejecución se aprobaron 19 de 20 validaciones. Los hallazgos y su corrección se resumen a continuación.

Tabla 10

Fallos encontrados y correcciones

N.º	Hallazgo	Cómo se detectó	Acción correctiva	Resultado
1	El nombre de la caneca aparecía en minúscula («blanca») en la etiqueta de los resultados de búsqueda.	La validación PV-02 falló en la primera ejecución.	Se agregó la función chipTexto(), que escribe la primera letra en mayúscula.	PV-02 aprobada.
2	En la sección «Las canecas» el nombre y la descripción quedaban pegados, y el aviso de recolección mostraba mal alineado su título y su botón.	Revisión visual de las capturas de pantalla.	Se agregó separación entre textos y el título del aviso pasó a mostrarse en bloque.	Se repitieron las validaciones: 20 de 20 aprobadas.

Nota. Elaboración propia.

Trazabilidad entre requerimientos y pruebas

Tabla 11

Matriz de trazabilidad

Requisito	Pruebas que lo verifican	Estado
RF-01	PU-01 a PU-05, PI-03, PV-02 a PV-04	Verificado
RF-02	PU-17, PU-18, PI-02, PV-05, PV-06, PV-14	Verificado
RF-03	PU-06, PU-07, PI-03	Verificado
RF-04	PU-08, PU-09, PV-07, PV-08, PV-10	Verificado
RF-05	PU-10 a PU-13	Verificado
RF-06	PU-14 a PU-16, PV-11, PV-12	Verificado
RF-07	Requiere un dispositivo real con permiso de notificaciones	[pendiente: probar en un celular]
RF-08	PI-01, PI-04, PV-13, PV-14	Verificado
RNF-02	PV-15 a PV-17	Verificado
RNF-03	Medición del peso total (40,4 KB)	Verificado
RNF-04	PV-18, PV-19	Verificado
RNF-01 y RNF-06	Revisión de diseño; falta validar con vecinos	[completar con la prueba de usabilidad]

Nota. Elaboración propia.

Prueba de usabilidad con vecinos

Para cumplir el objetivo específico 4 se hará una prueba de usabilidad con 5 a 10 vecinos voluntarios, a partir de los criterios de Nielsen (1993): facilidad de aprendizaje, eficiencia, facilidad para recordar, errores y satisfacción. Cada participante realizará tres tareas: (T1) averiguar dónde va una pila, (T2) configurar el día de recolección de su sector y (T3) jugar una ronda del quiz.

Tabla 12

Registro de la prueba de usabilidad

Participante	T1 (¿lo logró?)	T2 (¿lo logró?)	T3 (¿lo logró?)	Observaciones y satisfacción (1 a 5)
Vecino 1	[completar]	[completar]	[completar]	[completar]

Participante	T1 (¿lo logró?)	T2 (¿lo logró?)	T3 (¿lo logró?)	Observaciones y satisfacción (1 a 5)
Vecino 2	[completar]	[completar]	[completar]	[completar]
Vecino 3	[completar]	[completar]	[completar]	[completar]
Vecino 4	[completar]	[completar]	[completar]	[completar]
Vecino 5	[completar]	[completar]	[completar]	[completar]

Nota. Elaboración propia. Esta tabla se completa con los datos reales de la prueba.

Conclusiones de la validación

Las 19 pruebas de unidad, las 4 de integración y las 20 de validación pasan con éxito, por lo que el prototipo cumple los requerimientos funcionales verificados. Quedan dos limitaciones: aún no se probó el recordatorio (RF-07) en un celular real, y falta la prueba de usabilidad con vecinos, que aportará la retroalimentación para la siguiente iteración.

Referencias bibliográficas

Alcaldía de Popayán. (2016). *Plan de Gestión Integral de Residuos Sólidos – PGIRS 2016-2027*. https://www.popayan.gov.co/NuestraAlcaldia/Planeaciongestionycontrol/Informe%20de%20rendici%C3%B3n%20de%20cuentas%20%20PGIRS%202016.pdf

Asti Vera, A. (2015). *Metodología de la investigación*. Athenaica Ediciones Universitarias.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifiesto por el desarrollo ágil de software*. https://agilemanifesto.org/iso/es/manifesto.html

Booch, G., Rumbaugh, J., & Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2.ª ed.). Pearson Educación.

Clarke, R. I. (2020). *Design thinking*. ALA Neal-Schuman.

León, G. J. L., Socorro, C. A. R., & Espinoza, C. C. X. (2017). *Uso de la información científica y tecnológica en la investigación y la innovación*. [completar editorial o URL]

Lerma González, H. D. (2009). *Metodología de la investigación: propuesta, anteproyecto y proyecto* (4.ª ed.). Ecoe Ediciones.

Llamas Fernández, F. J. (2018). La metodología Lean Startup: Desarrollo y aplicación para el emprendimiento. *Revista Escuela de Administración de Negocios*, (84), 79-95. https://doi.org/10.21158/01208160.n84.2018.1918

Ministerio de Ambiente y Desarrollo Sostenible. (2016). *[[completar título del documento consultado]]*. [completar URL]

Ministerio de Ambiente y Desarrollo Sostenible. (2019). *Resolución 2184 de 2019: Por la cual se modifica la Resolución 668 de 2016 sobre uso racional de bolsas plásticas y se dictan otras disposiciones*. [completar URL]

Mozilla. (s. f.). *Progressive web apps*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/Progressive_web_apps

Naciones Unidas. (2015, 25 de septiembre). *La Asamblea General adopta la Agenda 2030 para el Desarrollo Sostenible*. https://www.un.org/sustainabledevelopment/es/2015/09/la-asamblea-general-adopta-la-agenda-2030-para-el-desarrollo-sostenible/

Narváez, L. C. (2016). *Esquema aprendizaje práctico para plantear un problema de investigación*. Repositorio Institucional UNAD. [completar URL]

Nielsen, J. (1993). *Usability engineering*. Academic Press.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

Villegas, E., Labrador, E., Fonseca, D., & Fernández-Guinea, S. (2018). Mejora de las metodologías de experiencia de usuario mediante la aplicación de gamificación: Metodología I'm In. [completar revista, volumen y páginas desde la ficha de la base de datos]

Bibliografía

Universidad Nacional Abierta y a Distancia. (2026a). *Guía de aprendizaje – Fase 2: Propuesta de proyecto aplicado* [Guía de actividad]. Campus virtual UNAD.

Universidad Nacional Abierta y a Distancia. (2026b). *Guía de aprendizaje – Fase 3: Implementación del proyecto* [Guía de actividad]. Campus virtual UNAD.

[agregar aquí otras obras consultadas que no se citen en el texto]